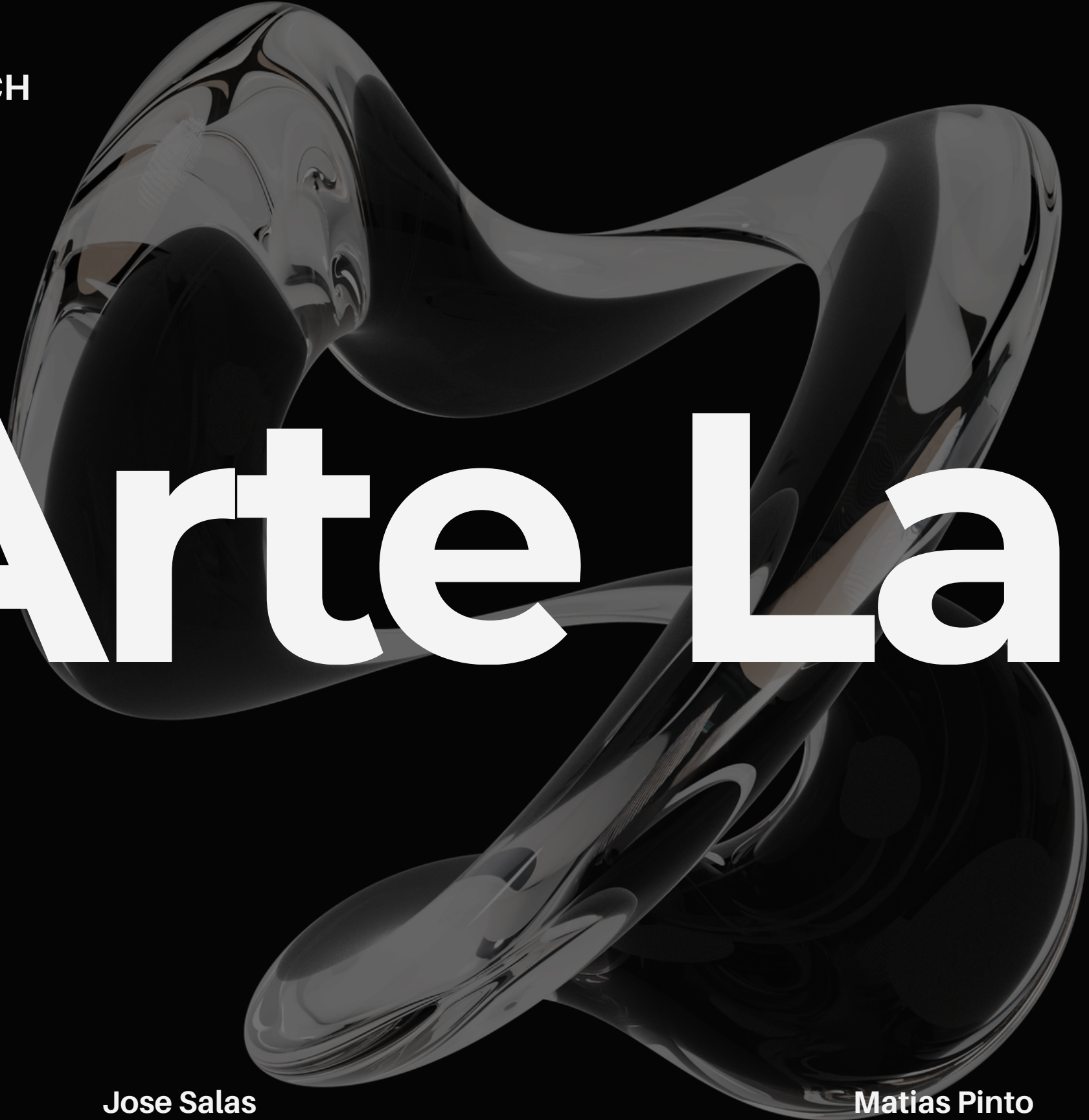


**Desarrollo Fullstack**

**ROBERTO SIMON OSSES VECCH**

Jul 14 del 2026

An abstract sculpture made of a dark, glossy, liquid-like material. It forms a shape that resembles a face, with a large, open mouth and flowing, undulating features. The sculpture is highly reflective, showing highlights and shadows that emphasize its fluid, organic form.

# Arte Lab

**Felipe Galdames**

**Jose Salas**

**Matias Pinto**

**Leonard Hernández**

# Introduccion

- Objetivo de la Presentación: Demostrar el dominio técnico, la autonomía y la capacidad de resolución en la construcción de la arquitectura de software para el Examen Transversal.
- El Proyecto ArteLab: Desarrollo de un sistema backend distribuido basado en una arquitectura de microservicios, destacando la interoperabilidad entre los servicios principales de artelab y usuarios.
- Diseño Arquitectónico: Implementación rigurosa del Patrón CSR (Controller, Service, Repository) para garantizar una separación limpia de responsabilidades lógicas, de exposición (HTTP) y de persistencia de datos.
- Pilares Técnicos del Ecosistema: Integración de persistencia relacional normalizada mediante entidades JPA, validaciones robustas de entrada, gestión centralizada de fallas y pruebas unitarias automatizadas.
- Enfoque de la Defensa: Validación en vivo del funcionamiento de los endpoints REST, flujos de comunicación distribuida y la capacidad de adaptar lógicas de negocio en tiempo real.



```
test\java\cl\artelab_spa\artelab
├── client
│   └── UsuarioClientTest.java
├── controller
│   ├── AuthControllerTest.java
│   ├── CategoriaControllerTest.java
│   ├── ProductoControllerTest.java
│   └── PromocionControllerTest.java
├── service
│   ├── CategoriaServiceTest.java
│   ├── ProductoServiceTest.java
│   ├── PromocionServiceTest.java
│   └── ArtelabApplicationTests.java
```

# Pruebas Unitarias Funcionales (JUnit & Mockito)

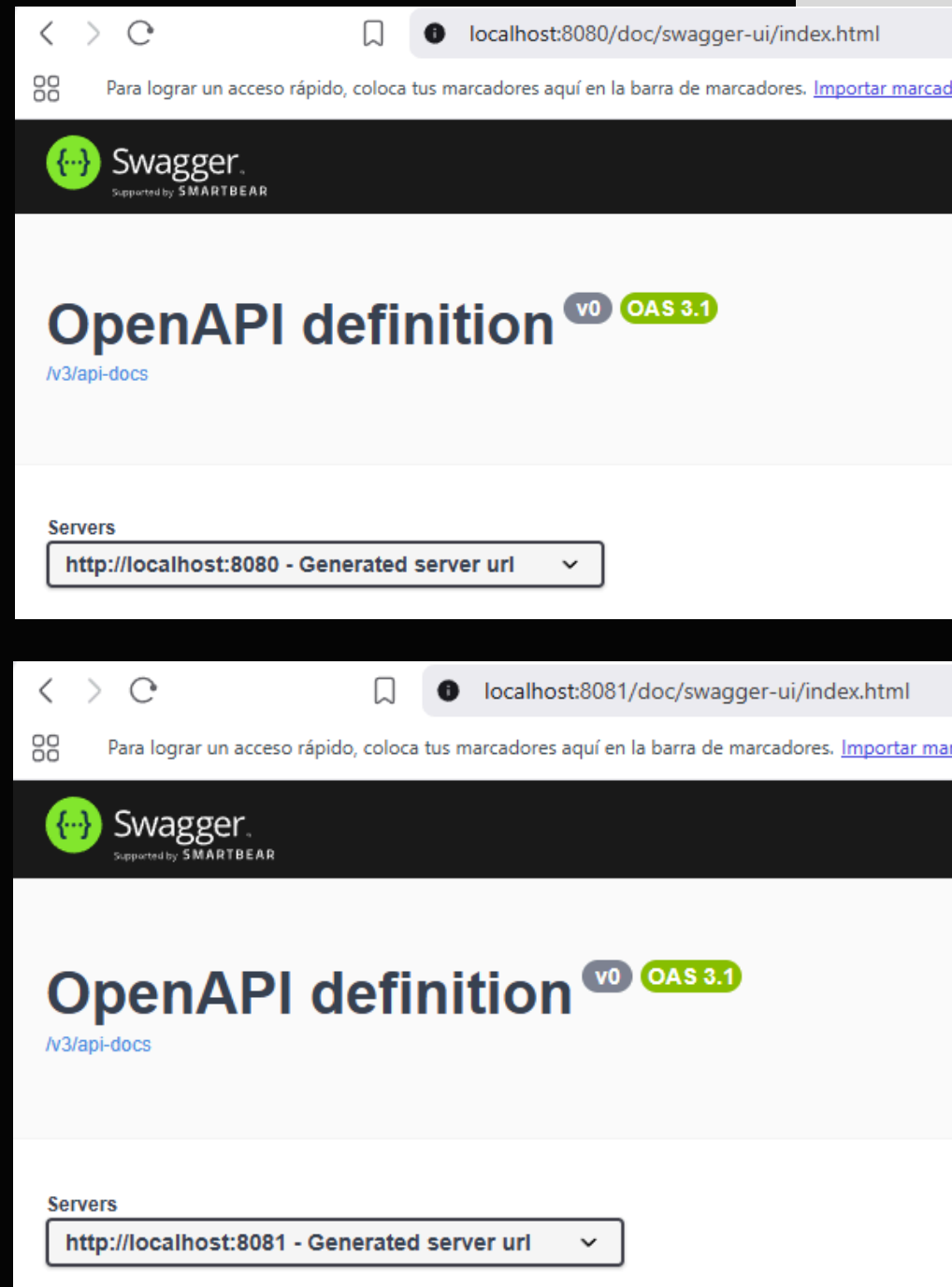
- Clases Principales: UsuarioServiceTest, ProductoServiceTest y PromocionServiceTest.
- Simulación de Entorno (@Mock): Simula repositorios y clientes remotos sin requerir conexión a la base de datos real.
- Preparación de Escenarios: Uso de when(...).thenReturn(...) para definir el comportamiento esperado del test.
- Validación de Resultados: assertEquals(...) para comprobar datos correctos y assertThrows(...) para validar excepciones esperadas (ej. duplicados o reglas inválidas).
- Verificación de Ejecución: verify(...) confirma que los métodos como save o delete fueron llamados correctamente.



- > assemblers
- > client
- > config
- > controller
- > data
- > dto
- > exception
- > model
- > repository
- > security
- > service

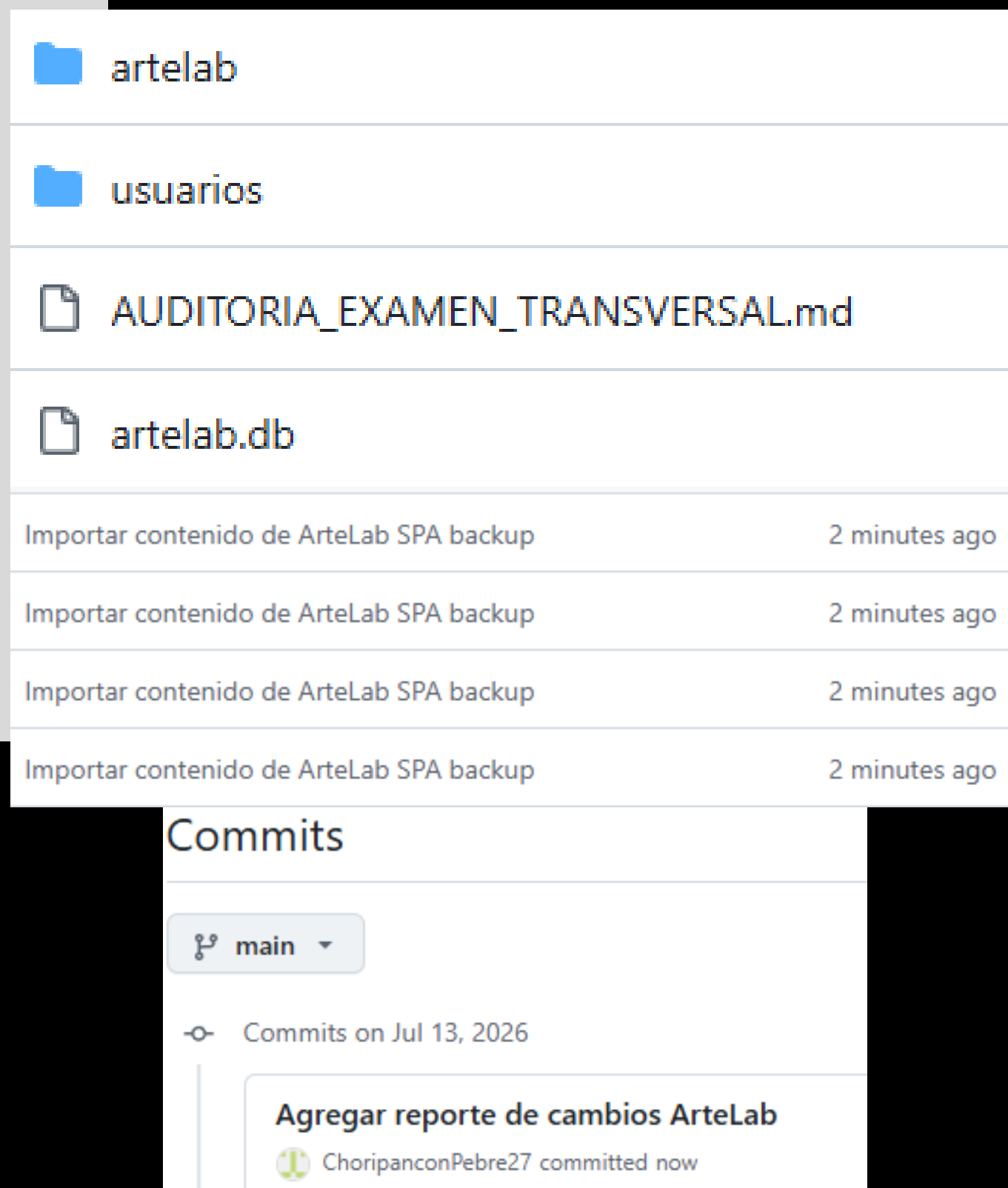
# Arquitectura CRUD y Reglas de Negocio

- Separación de Responsabilidades: El controlador solo recibe peticiones HTTP y delega; la capa Service decide la existencia y las reglas de negocio.
- Alteración de Reglas CRUD: Cualquier cambio en la lógica se modifica directamente en los Services (ProductoService, CategoriaService, UsuarioService).
- Validaciones de Entrada: Las restricciones simples se revisan directamente en el modelo/DTO.
- Respuestas de Error HTTP: Las modificaciones de códigos de estado se gestionan en



# Ejecución Autónoma y Documentación API

- Orden de Levantamiento: Inicialización del microservicio usuarios primero, seguido de artelab configurado para apuntar a usuarios.
- Configuración Remota: La URL base (usuarios.service.base-url) y los timeouts se configuran en WebClientConfig.
- Validación Documental: Verificación de endpoints autogenerados en la ruta Swagger /doc/swagger-ui.html.
- Ejecución Vía Consola: Uso de comandos nativos mvnw.cmd verify para pruebas y mvnw.cmd spring-boot:run para levantar el entorno.



# Evidencia de Aporte Personal Colaborativo

- Control de Versiones: Historial de commits propios y archivos directamente modificados en el repositorio de GitHub.
- Gestión de Tareas: Tarjetas y actividades asignadas al usuario dentro del tablero de Trello.
- Documentación de Auditoría: Evidencia respaldada en los archivos locales AUDITORIA\_EXAMEN\_TRANSVERSAL.md y REPORTE\_CAMBIOS\_ARTE\_LAB.md.
- Áreas de Impacto Demostrables: Desarrollo en capas Service, validaciones, manejo global de errores, pruebas unitarias y documentación técnica.

▼ exception

J BusinessException.java

J GlobalExceptionHandler.java

J RemoteServiceException.java

J ResourceConflictException.java

▼ service

J CategoriaService.java

J ProductoService.java

J PromocionService.java

▼ client

J UsuarioClient.java

# Gestión de Fallas y Códigos HTTP

- Manejo Centralizado: Intercepción de errores a través de la clase GlobalExceptionHandler.java.
- Recursos No Encontrados: EntityNotFoundException devuelve un código HTTP 404.
- Infracción de Lógica: Violaciones a las reglas de negocio devuelven un código HTTP 400.
- Conflictos de Datos: Errores de duplicidad o conflictos devuelven un código HTTP 409.
- Fallas del Sistema: Errores remotos devuelven HTTP 503 y errores internos devuelven HTTP 500 de manera controlada.

```
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    private Map<String, Object> buildErrorBody(HttpStatus status, String error, String message) {
        Map<String, Object> body = new HashMap<>();
        body.put(key: "timestamp", LocalDateTime.now().toString());
        body.put(key: "status", status.value());
        body.put(key: "error", error);
        body.put(key: "message", message);
        return body;
    }

    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleEntityNotFound(EntityNotFoundException ex) {
        log.warn("Entity not found: {}", ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(buildErrorBody(HttpStatus.NOT_FOUND, error: "Not Found", ex.getMessage()));
    }
}
```

```
@Service
public class UsuarioClient {

    private static final Logger log = LoggerFactory.getLogger(UsuarioClient.class);

    private final WebClient usuariosWebClient;

    public UsuarioClient(WebClient usuariosWebClient) {
        this.usuariosWebClient = usuariosWebClient;
    }

    public Optional<UsuarioLookupDto> findUsuarioById(Long id) {
        try {
            return usuariosWebClient.get()
                .uri("/api/v1/usuarios/{id}/lookup", id)
                .exchangeToMono(this::handleResponse)
                .block();
        } catch (RemoteServiceException ex) {
            log.error("Remote usuarios service failure for id={}", id, ex);
            throw ex;
        } catch (Exception ex) {
            log.error("Unexpected error calling usuarios service for id={}", id, ex);
            throw new RemoteServiceException(message: "Error connecting to usuarios service", ex);
        }
    }
}
```

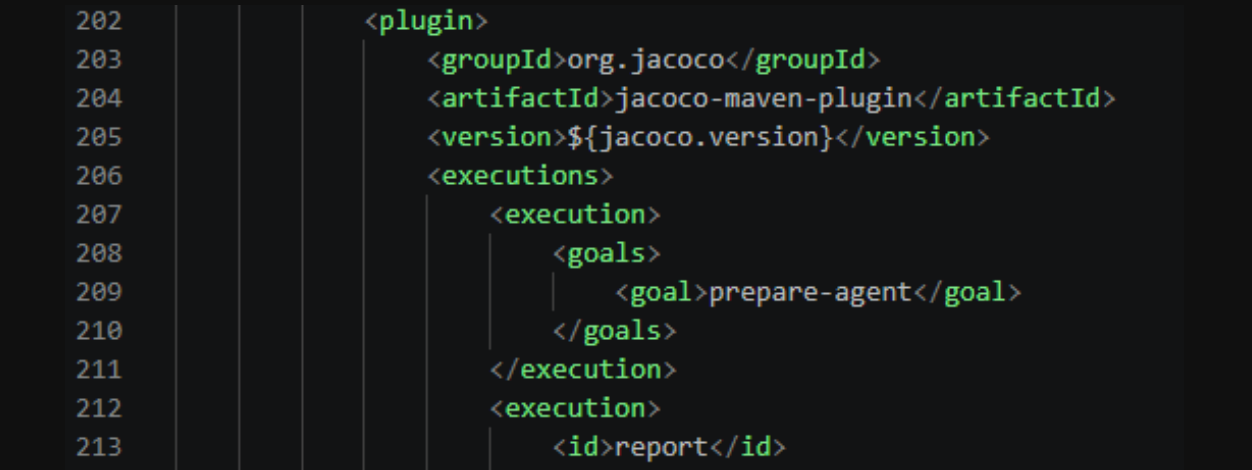
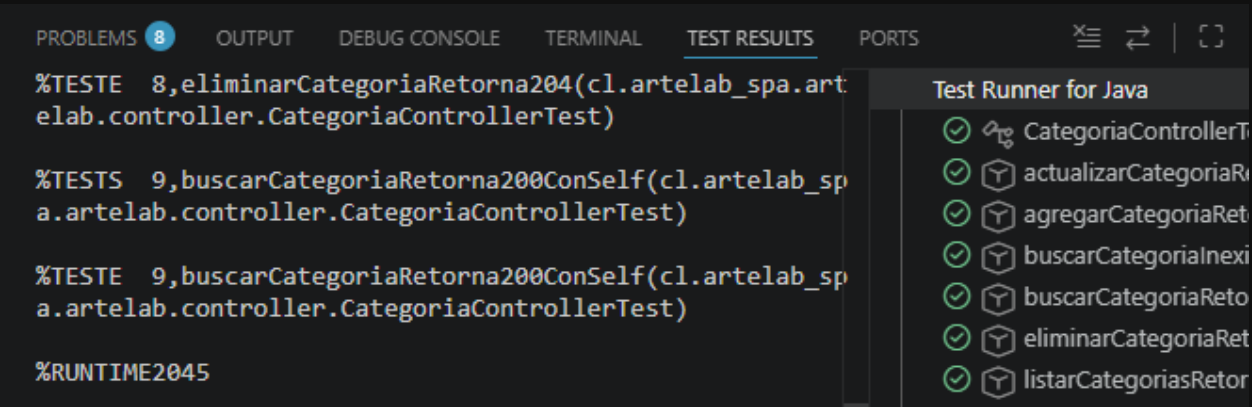
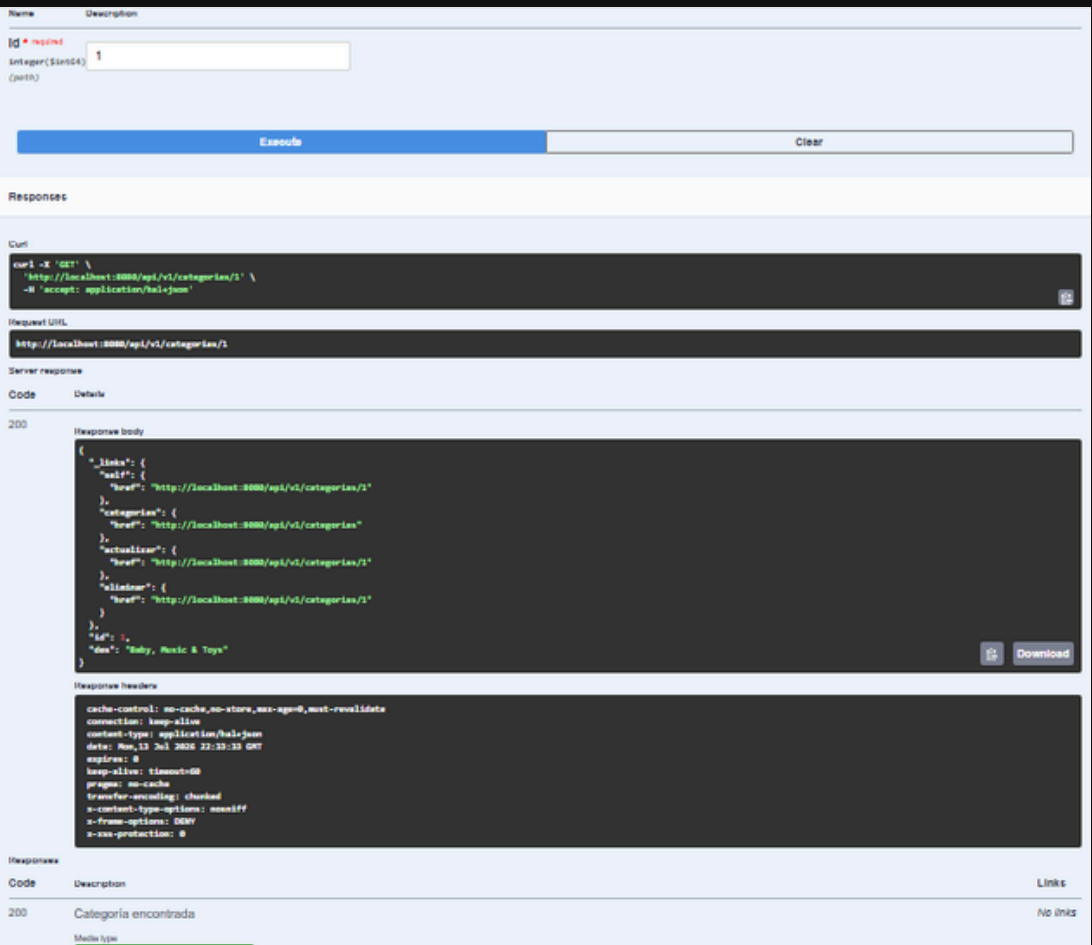
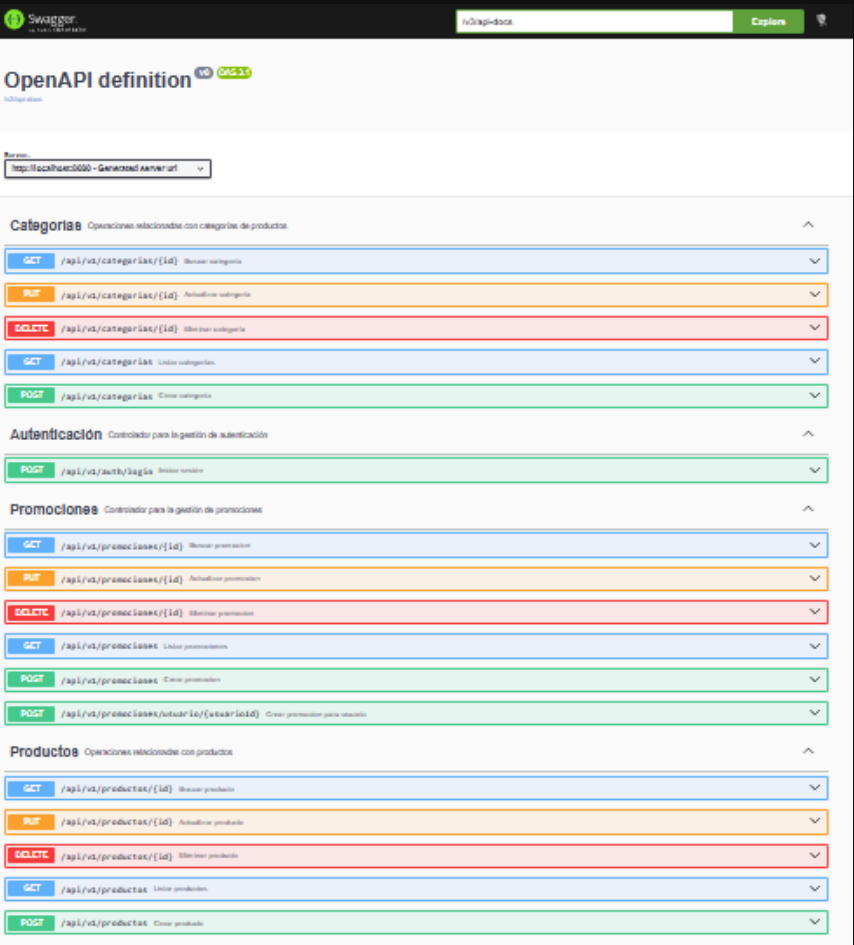
# Intercomunicación de Microservicios (Flujo Distribuido)

- Consulta Externa: El microservicio artelab no inventa datos; consulta la existencia de usuarios directamente al microservicio usuarios.
- Cliente REST: La comunicación remota está encapsulada utilizando UsuarioClient basado en WebClient.
- Manejo de Errores 404: Si el servicio de usuarios responde 404, se procesa formalmente como usuario no encontrado.
- Gestión de Caídas (5xx): Si el servicio remoto falla o pierde conexión, se lanza una RemoteServiceException que se traduce al cliente final como HTTP 503.



# Estado Final

El proyecto ya cumple de forma sólida los requisitos técnicos más importantes del examen transversal: arquitectura por capas, validaciones, manejo centralizado de errores, comunicación entre microservicios, Swagger/OpenAPI, pruebas unitarias y cobertura con JaCoCo. Los cambios realizados dejaron los microservicios más consistentes, más fáciles de mantener y mejor preparados para una evaluación técnica.





**Gracias por  
su atención**